

AGRO GESTÃO · TERRITÓRIO OS

# Diretrizes de Linguagem e Fala

Super dicionário, gramática, previsão de palavras e voz para o lançamento de vendas dos produtores rurais

Documento técnico para integração via Claude Code no APK

**Alex Duvier**

Carangola · Minas Gerais · versão 1.0.0 · 19 de outubro de 2026

**Para:** Claude Code (integração no app/APK AGRO Gestão — Território OS)

**Autor:** Alex Duvier · **Versão:** 1.0.0 · **Data:** 19/10/2026

**Arquivos deste pacote:** `dicionario_agro_ptbr.json`, `motor-linguagem.js`, `testes.mjs`, `gerar_dicionario.py`, `integracao/`, `teclado/`

> *Objetivo: o produtor fala ou digita do jeito dele ("vemdi tres cafe especial a 25 no pics") e o app registra exatamente "Vendi 3 cafés especiais a R$ 25,00 cada, no Pix." — com a precisão de um assistente de voz de ponta, funcionando também sem internet.*

# 0. ANTES DE QUALQUER CÓDIGO: localizar a memória de produtos do app (obrigatório)

O app **já tem uma memória com mais de 2 mil produtos**. Ela é a **fonte primária**. O dicionário deste pacote é apenas **semente** (gramática, erros, unidades, numerais e ~180 produtos típicos da Zona da Mata).

Claude Code, execute nesta ordem e **relate o que encontrou antes de alterar qualquer arquivo**:

1. **Mapeie a stack** do projeto: `package.json` (React? Vite? Capacitor? React Native? Expo? Cordova? TWA/PWA?), pasta `android/`, `capacitor.config.*`, `app.json`.
2. **Encontre o catálogo de produtos.** Procure por:
   - arquivos: `produtos*.json`, `catalogo*.json`, `products*.json`, `*.csv`, `seed*`, `fixtures*`, `assets/data/*`;
   - código: `grep -ri "produto\|product\|catalog\|catalogo\|estoque\|inventory"` em `src/`;
   - banco local: SQLite (`@capacitor-community/sqlite`, `expo-sqlite`, `react-native-sqlite`), IndexedDB/Dexie, `localStorage`, Capacitor `Preferences`;
   - nuvem: chamadas ao **Cloudflare** (Workers, D1, KV, R2) — o app envia relatórios da feira para lá.
3. **Descreva o esquema** encontrado (nome dos campos de id, nome, categoria, unidade, preço, produtor/expositor, apelidos) e a **contagem real** de produtos.
4. **Verifique se já existe** algum corretor, autocomplete, dicionário, `normalize`, `fuse.js`, `levenshtein`, reconhecimento de voz (`SpeechRecognition`, `@capacitor-community/speech-recognition`, `react-native-voice`, Whisper, OpenAI) — **não duplique**: integre ou substitua com aprovação do Alex.
5. **Só então** integre conforme a seção 3, chamando `motor.carregarCatalogo(listaDoApp, mapeamentoDeCampos)`.

Se o catálogo estiver só no servidor (Cloudflare D1/KV), crie uma **cópia local sincronizada** (o motor precisa funcionar offline na praça, onde o sinal costuma cair).

# 1. O que o motor faz

| Função | O que entrega |
|---|---|
| `limparEntrada(texto)` | Remove o "lixo" de teclados chineses e ditado: letras/números de largura total (`３ｋｇ` → `3kg`), pontuação CJK (`，` `。` `￥`), espaços ideográficos, caracteres invisíveis, emojis; normaliza acentos (NFC). |
| `motor.interpretar(texto)` | Lançamento estruturado: ação, itens (produto, quantidade, unidade, preço unitário/total), pagamento (inclusive dividido), total, **frase corrigida**, correções feitas, **confiança (0–1)** e **pendências** a confirmar. |
| `motor.prever(texto)` | Sugestões para a barra acima do teclado: completa a palavra ("vendi 3 cafe es" → **cafés especiais**, já no plural) ou prevê a próxima ("no " → Pix, dinheiro, cartão; "doce de " → leite, abóbora…; após produto → último preço praticado). |
| `motor.corrigirTexto(texto)` | Correção de texto livre (feedback, observações) **preservando o original** ao lado. |
| `motor.aprender(resultado)` | Memória do produtor: produtos mais vendidos, último preço, forma de pagamento preferida, grafias pessoais. Chamar **só após confirmação**. |
| `motor.ensinarApelido("pdq", id)` | Atalhos pessoais do produtor. |
| `motor.gerarKeywordsVoz()` / `gerarPromptVoz()` | Lista de palavras-chave e prompt de contexto para o reconhecimento de voz (seção 4). |
| `motor.fraseParaFalar(resultado)` | Frase para ler em voz alta na confirmação, com números por extenso e gênero correto ("duas goiabadas a doze reais cada"). |

Exemplos reais (todos cobertos por `testes.mjs`, 50/50 aprovados):

| Produtor digitou/falou | Motor registra |
|---|---|
| vendi tres cafe especial a 25 no pics | Vendi 3 cafés especiais a R$ 25,00 cada, no Pix. |
| vemdi 2 quejo meia cura por 60 em dinhero | Vendi 2 queijos meia cura por R$ 60,00, em dinheiro. |
| sobrou meia duzia de ovo caipira | Sobrou meia dúzia de ovos caipiras. |
| vendi 2 pao de queijos e 1 doce de leite no cartao | Vendi 2 pães de queijo e 1 doce de leite, no cartão. |
| vendi uma duzia e meia de banana 15 reais no dinheiro | Vendi 1,5 dúzia de bananas por R$ 15,00, em dinheiro. |
| vendi 4 cafe gurme a 30 cada 2 no credito e 2 no pix | Vendi 4 cafés gourmet a R$ 30,00 cada, sendo 2 no cartão de crédito e 2 no Pix. |
| vende 2 bolo de fuba e 1 de cenoura no pix | Vendi 2 bolos de fubá e 1 bolo de cenoura, no Pix. |
| VENDI 3 CAFÉ ESPECIAL, NO PIX (teclado chinês) | Vendi 3 cafés especiais, no Pix. |
| vendi um doce de abobra e duas cocada 20 reais e cinquenta centavos dinheiro | Vendi 1 doce de abóbora e 2 cocadas por R$ 20,50, em dinheiro. |

Desempenho medido (Node 22): 0,2 ms por frase; previsão em ~3 ms com 2.180 produtos; carga de 2 mil produtos em ~0,2 s.

# 2. Regras e diretrizes de linguagem (o "contrato" do app)

## 2.1 Princípios

1. **Registro estruturado sempre corrigido; voz do produtor sempre preservada.** O lançamento (produto, quantidade, preço) usa a forma padrão; depoimentos e feedbacks guardam o texto original **e** a versão corrigida (regra R16). Nunca "corrigir" o sotaque de uma citação que vai para o relatório.
2. **Nunca gravar sem confirmação** quando `confianca < 0,85` ou houver `pendencias`. Mostrar um cartão de confirmação com a frase corrigida e botões grandes: **Confirmar · Corrigir · Cancelar**.
3. **Nunca inventar preço.** Se o produtor não disse, o campo fica vazio; o motor pode **sugerir** o último preço (`precoSugerido`), mas o produtor confirma.
4. **Ambiguidade vira pergunta curta, nunca palpite silencioso**: "R$ 60,00 é o valor TOTAL dos queijos?" / "3 é a QUANTIDADE de pão de queijo (e não o preço)?".
5. **O catálogo do produtor vem primeiro** (`definirProdutor(id)`): seus produtos têm prioridade na previsão e na voz.
6. **Offline primeiro.** Tudo do motor roda no aparelho; a voz em nuvem é um reforço, não uma dependência.

## 2.2 Gramática aplicada (resumo de `regras_gramaticais` no JSON)

- **R01 Plural após numeral:** ≥ 2 → plural; entre 1 e < 2 (1; 1,5) → singular. *vendi tres cafe especial → Vendi 3 cafés especiais.*
- **R02 Adjetivo concorda:** cafés especial → cafés especiais; ovos caipira → ovos caipiras.
- **R03 Composto com preposição:** só o núcleo flexiona: pão de queijos → pães de queijo; doces de leites → doces de leite.
- **R04 Composto com hífen:** batatas-doces, cheiros-verdes, couves-flores.
- **R05 Gênero do numeral:** duas goiabadas, dois queijos, meia dúzia, meio quilo.
- **R06 Unidades:** por extenso concorda (2 quilos, 3 dúzias); símbolo não tem plural nem ponto (2 kg, 500 g, 2 L).
- **R07 Produto após unidade:** coletiva + contável → plural (2 dúzias de bananas); embalagem → singular (3 pacotes de café especial).
- **R08 Moeda:** `R$ 25,00` (espaço, vírgula decimal, ponto de milhar).
- **R09 Crase:** pagou **à vista**; a prazo (sem crase).
- **R10 Pix:** sempre "Pix" (grafia do Banco Central), nunca "PIX" ou "pics".
- **R11 Acordo Ortográfico de 2009:** geleia, ideia, linguiça, pé de moleque, pão de queijo (sem hífen).
- **R12–R15:** "faz dois dias", "houve", "vendem-se ovos", horas "9h30", meses e dias em minúscula.

## 2.3 Como a correção decide (ordem, da mais segura para a menos)

1. Tabela de erros conhecidos (`erros`, 110 entradas: ortografia, fala mineira, acordo, contexto, concordância).
2. Vocabulário exato (produtos do catálogo + semente, unidades, numerais, ações, pagamentos, palavras funcionais).
3. Abreviações de WhatsApp (`vc`, `pct`, `dz`, `qtd`, `conto` = reais...).

4. **Chave fonética pt-BR** (quejo/kejo/queju → queijo; xuxu → chuchu; mio → milho; especiau → especial).
5. **Distância de edição ponderada pelo teclado** (letras vizinhas no QWERTY custam menos; letra dobrada e troca de posição são baratas).
6. Se nada for confiável: a palavra fica como está e vira pendência — **nunca trocar por algo distante**.

### 2.4 Gatilhos com armadilhas (já tratados — não "corrigir" de volta)

- "**caiu no Pix**" = pagamento recebido, **não** é perda.
- "**comprou / levou**" (na boca do produtor) = o cliente comprou = **venda**.
- "**meio**" (meio quilo) não é "mel", embora soem parecido.
- "**no**" pode ser "nó" (interjeição mineira) — só vira marcador de pagamento quando seguido de Pix/cartão/dinheiro.
- "**molho**" pode ser unidade (molho de cebolinha = maço) ou produto (molho de tomate): o produto composto tem prioridade.
- Número logo após o produto sem quantidade ("queijo 3"): até 12 → quantidade (com confirmação); acima disso → preço.
- "**a** 25" = preço unitário; "**por** 70" ou valor solto = total; "tudo/total/ao todo" = total da venda.

### 2.5 Previsão de palavras (barra de sugestões)

A frase de lançamento segue o molde: `[AÇÃO] [QUANTIDADE] [UNIDADE] [de] [PRODUTO] [VARIEDADE] [PREÇO] [cada|o kg|total] [PAGAMENTO]`, repetível com "e" ou vírgula. O motor sabe em que "casa" do molde o produtor está e sugere só o que cabe ali, já flexionado. Ranking: casa do molde 40%, histórico do produtor 30%, frequência geral 15%, recência 10%, catálogo do app 5% (pesos em `regras_previsao.pesos_ranking`).

- Mostrar no máximo **5 sugestões**, em chips grandes (mínimo 48 dp de altura), acima do teclado.
- Tocar no chip **substitui** o trecho indicado em `substituir.inicio–substituir.fim` (o motor pode trocar "cafe es" inteiro por "cafés especiais").
- Debounce de **80 ms** na digitação.

## 3. Integração no app (passo a passo para o Claude Code)

### 3.1 Arquivos

Copie para o projeto (ajuste ao padrão de pastas existente):

```
src/linguagem/motor-linguagem.js
src/linguagem/dicionario_agro_ptbr.json
src/linguagem/testes.mjs          (rodar com: node src/linguagem/testes.mjs)
```

### 3.2 Inicialização (uma vez, no boot do app)

```
import dic from './linguagem/dicionario_agro_ptbr.json';
import { criarMotor } from './linguagem/motor-linguagem.js';

export const motor = criarMotor(dic, {
  // Persistência: prefira o armazenamento que o app JÁ usa (SQLite / Preferences / IndexedDB).
  armazenamento: { get: (k) => storage.getSync(k), set: (k, v) => storage.setSync(k, v) },
});
const produtos = await carregarProdutosDoApp();          // ← a memória de 2 mil+ encontrada na seção 0
motor.carregarCatalogo(produtos, { id: 'id', nome: 'nome', categoria: 'categoria',
  unidade: 'unidade', produtor: 'expositorId', apelidos: 'apelidos' });
motor.definirProdutor(usuarioLogado.id);
```

Se o armazenamento do app for assíncrono, carregue a memória com `motor.importarMemoria(await storage.get('agro_gestao_linguagem_v1'))` e salve com `exportarMemoria()` após cada `aprender`.

### 3.3 Fluxo do lançamento

1. Campo de texto (ou voz → texto) → `motor.interpretar(texto)`.
2. Se `confianca ≥ 0,85` e sem `pendencias`: mostrar o cartão já pronto para confirmar com um toque.
3. Caso contrário: mostrar o cartão com as pendências destacadas e campos editáveis (produto, quantidade, preço, pagamento).
4. Ao confirmar: gravar na seção de **vendas** (estoque do evento + horário — é dela que saem os relatórios Verde e Azul), depois `motor.aprender(resultado, { textoDigitado })`.
5. Mapear `acao` → operação: `venda`, `fiado` (venda a prazo), `recebimento`, `sobra`, `perda`, `brinde`, `troca`, `reposicao`, `estoque_entrada`, `cancelar` (desfaz o último lançamento, sempre com confirmação).
6. Usar `quantidadeBase` (dúzia × 12) para baixar estoque em unidades.

### 3.4 Campo de digitação (essencial em celular chinês)

Exemplo completo em `integracao/CampoLancamento.jsx`. Regras:

```
<input lang="pt-BR" inputmode="text" enterkeyhint="done"
       autocomplete="off" autocorrect="off" spellcheck="false" autocapitalize="sentences">
```

- **Composição de IME:** teclados chineses (Sogou, Baidu, iFlytek, teclados de fábrica OPPO/Xiaomi/vivo) e até o Gboard enviam texto "em composição". **Não processe** enquanto `event.isComposing === true` ou `keyCode === 229`; processe no `compositionend` e no `input` normal.
- Desligar o corretor do teclado no campo evita que ele "brigue" com as sugestões do app.
- Passar todo texto por `limparEntrada()` antes de salvar qualquer coisa (inclusive nomes de produtos e clientes).

### 3.5 Testes obrigatórios antes de gerar o APK

- `node src/linguagem/testes.mjs` → 50 ok, 0 falha.
- Acrescentar ao `testes.mjs` **20 frases reais por produtor** da próxima Feira Mensal (copiar do histórico de vendas ou gravar com eles) e manter a taxa de acerto ≥ 95% no lançamento final (após confirmação).
- Teste manual no **OPPO** (Reno 16 e Find X8 Ultra): Gboard, teclado de fábrica e ditado por voz.

# 4. Voz com precisão de nível "GPT voz"

Precisão em feira (barulho, sotaque mineiro, nomes de produtos) não vem de um único reconhecedor: vem de **três camadas**. Pesquisa feita em 19/10/2026, priorizando o que foi atualizado recentemente.

## 4.1 Camada 1 — Reconhecimento com vocabulário do produtor (nuvem, maior precisão)

**Recomendado: OpenAI** `gpt-transcribe` / `gpt-live-transcribe` (lançados em 28/07/2026). São os primeiros modelos de transcrição da OpenAI com campo próprio de **palavras-chave** (`keywords`) além do `prompt` de contexto e da lista `languages`.

- **Gravação curta (apertar e falar, recomendado para a feira):** enviar o áudio a `gpt-transcribe` com `languages: ["pt"]`, `keywords: motor.gerarKeywordsVoz()` e `prompt: motor.gerarPromptVoz()`.
- **Ao vivo (texto aparecendo enquanto fala):** sessão Realtime de transcrição com `gpt-live-transcribe`, áudio PCM 24 kHz, `delay` entre `minimal` e `xhigh` (mais atraso = mais precisão). Esse modelo não faz detecção de fim de fala no servidor: o app detecta o silêncio e envia o *commit*.
- **Keywords:** só os produtos do produtor + Pix/débito/crédito/dúzia (o motor limita a 60). **Não mande os 2 mil:** keywords demais ou irrelevantes fazem o modelo "ouvir" palavras não ditas. Sem <, > nem quebras de linha.
- **Chave da API NUNCA dentro do APK.** O app envia o áudio para um **Cloudflare Worker** (o projeto já usa Cloudflare) que guarda a chave como *secret* e repassa à OpenAI. Exemplo em `integracao/transcrever-worker.js` — **confirmar nomes de campos e preços na documentação oficial no dia da implementação**, porque esses modelos são recentes.
- Alternativas com vocabulário próprio, caso a OpenAI não sirva: **Google Cloud Speech-to-Text Chirp 3** (pt-BR GA, "speech adaptation" com até 1.000 frases, streaming); Azure Speech (Phrase List); AWS Transcribe (vocabulário personalizado pt-BR). O Deepgram Nova-3 tem *keyterm prompting*, mas o material consultado não confirma suporte a português nesse recurso.

## 4.2 Camada 2 — Reconhecimento no aparelho (offline, gratuito)

- **Android SpeechRecognizer (Google)** via plugin `@capacitor-community/speech-recognition` (se o app for Capacitor) com `language: 'pt-BR'` e `partialResults: true`. No OPPO, instalar/atualizar **"Serviços de fala do Google"** e baixar o pacote offline de português (Brasil). Atenção: a lista de idiomas do plugin não funciona no Android 13+; fixe `pt-BR`.
- **whisper.cpp** (Whisper rodando no próprio celular) — opção para quem quer offline com qualidade; o Whisper large-v3 tem WER de cerca de 8% em português brasileiro no Common Voice (fonte secundária), mas é pesado para celular; modelos *small/medium* quantizados são o meio-termo.
- **sherpa-onnx** (k2-fsa, versão 1.13.7 de 01/09/2026): ótimo motor offline com *hotwords*, porém hotwords só funcionam com modelos *transducer* — e não há transducer de português brasileiro de destaque; trate como pesquisa futura.
- **Vosk** (modelo pequeno pt, 31 MB): leve, mas WER alto (32,6% no Common Voice e 68,9% no CORAA). Útil apenas como "modo comando" com gramática restrita (lista fechada de palavras), não para fala livre.

### 4.3 Camada 3 — Motor de linguagem (sempre)

Qualquer texto vindo da voz passa por `motor.interpretar()`. É aqui que "três café especial" vira "3 cafés especiais", "caiu no Pix" vira pagamento e "quejo" vira queijo — a mesma etapa que a própria documentação da OpenAI recomenda (pós-processar a transcrição com um modelo de texto). Opcional: para frases com `confianca < 0,6`, enviar texto + lista de produtos do produtor a um LLM com saída estruturada (JSON) no Worker, e mostrar o resultado para confirmação.

### 4.4 Confirmação por voz

Depois de interpretar, o app pode **ler em voz alta** `motor.fraseParaFalar(resultado)` ("Vendi três cafés especiais a vinte e cinco reais cada, no Pix") usando o TTS do aparelho em pt-BR, e aceitar "sim/confirma" ou "não/corrige".

### 4.5 Boas práticas de áudio na praça

Botão **apertar-e-falar** (evita captar conversa alheia); microfone a 10–20 cm da boca; 16 kHz mono ou mais; cortar silêncio; frases curtas ("vendi dois queijo no Pix"). Ensinar o produtor com 3 exemplos no primeiro uso.

## 5. Celular chinês (OPPO / ColorOS) — checklist

1. **Teclado:** Gboard em português (Brasil) é o mais previsível. Se o aparelho vier com teclado de fábrica ou Sogou/Baidu, o app continua funcionando porque `limparEntrada()` e o tratamento de composição (3.4) cobrem os casos.
2. **Dicionário pessoal do teclado:** o pacote traz `teclado/gboard_dicionario_pessoal.zip` com ~400 palavras e atalhos ("pdq" → pão de queijo, "cfe" → café especial). Importar em Gboard → Configurações → Dicionário → Dicionário pessoal → Português (Brasil) → menu ⋮ → Importar. Isso melhora também o WhatsApp e outros apps do produtor. Testar a importação antes de distribuir.
3. **Voz do sistema:** usar "Digitação por voz do Google" em pt-BR, não o assistente de fábrica (pode estar em chinês/inglês).
4. **Bateria:** ColorOS fecha apps em segundo plano. Para sincronizar vendas com o Cloudflare, pedir ao produtor que marque o AGRO Gestão como "Permitir atividade em segundo plano" e salvar tudo localmente antes de enviar.
5. **Permissões:** `RECORD_AUDIO` no `AndroidManifest.xml` e pedido de permissão na primeira vez que tocar no microfone; `INTERNET` para a camada de nuvem.
6. **WebView:** se o app for híbrido, manter o "Android System WebView" atualizado pela Play Store (o motor usa recursos de JavaScript modernos).

## 6. Manutenção do dicionário

- Editar `gerar_dicionario.py` (produtos-semente, unidades, numerais, ações, pagamentos, erros, regras) e rodar `python3 gerar_dicionario.py` para gerar o JSON.
- A cada feira, exportar `motor.exportarMemoria()` de cada aparelho (grafias aprendidas e apelidos) e consolidar as recorrentes no dicionário-semente.
- Toda mudança: rodar `node testes.mjs` antes de gerar o APK.
- Não adicionar como gatilho palavras de duplo sentido sem testar (ver 2.4).

## 7. Privacidade

- A memória do motor guarda produtos, preços e grafias — **não** guarda nome, CPF ou telefone de clientes.
- Áudio enviado à nuvem: não armazenar no Worker; repassar e descartar. Informar o produtor na primeira vez que usar a voz.

## 8. Fontes consultadas (19/10/2026)

- OpenAI — Guia Speech-to-text: https://developers.openai.com/api/docs/guides/speech-to-text
- OpenAI — Realtime transcription: https://developers.openai.com/api/docs/guides/realtime-transcription
- OpenAI — Novos modelos de voz na API (07/05/2026): https://openai.com/fr-CA/index/advancing-voice-intelligence-with-new-models-in-the-api/
- Knightli — GPT Transcribe vs. GPT Live Transcribe (lançamento em 28/07/2026): https://knightli.com/en/2026/07/31/gpt-transcribe-live-transcribe-speech-to-text-guide/
- Google Cloud — Chirp 3: https://docs.cloud.google.com/speech-to-text/docs/models/chirp-3
- Deepgram — Nova-3 e keyterm prompting multilíngue: https://deepgram.com/learn/deepgram-expands-nova-3-with-10-new-languages-and-multilingual-keyterm-prompting
- k2-fsa/sherpa-onnx — Releases: https://github.com/k2-fsa/sherpa-onnx/releases
- react-native-sherpa-onnx — Hotwords: https://mintlify.wiki/xdcobra/react-native-sherpa-onnx/guides/hotwords
- Vosk — Modelos: https://alphacephei.com/vosk/models
- capacitor-community/speech-recognition: https://github.com/capacitor-community/speech-recognition
- NovaScribe — Precisão do Whisper em pt-BR (jul/2026): https://novascribe.ai/pt/como-precisa-e-o-whisper
- Hugging Face — Whisper large-v3 pt (fine-tune): https://huggingface.co/fsicoli/whisper-large-v3-pt-cv16